

WAN NURAL JAWAHIR HJ WAN YUSSOF, RABIEI MAMAT,
MOHD ARIZAL SHAMSIL MAT RIFIN
& MUHAMMAD SYAFFIQ HAZARI

Contents

List of Figures	v
1 DEVELOPMENT ENVIRONMENT SETUP FOR JAVA WEB APPLICATIONS	1
1.1 Objectives	1
1.2 Introduction	1
1.3 Software Requirements	2
1.4 Task 1: Install Java Development Kit (JDK)	2
1.4.1 Step 1: Download JDK	2
1.4.2 Step 2: Install JDK	2
1.4.3 Step 3: Verify JDK Installation	2
1.5 Task 2: Install XAMPP	3
1.5.1 Step 1: Download XAMPP	3
1.5.2 Step 2: Install XAMPP without Tomcat	4
1.5.3 Step 3: Verify XAMPP Installation	9
1.6 Task 3: Change the Default Root Password of MySQL	11
1.6.1 Step 1: Start Apache and MySQL Services	11
1.6.2 Step 2: Run Command	12
1.7 Task 4: Tomcat 9 Installation	18
1.7.1 Step 1: Download Apache Tomcat 9	18
1.7.2 Step 2: Extract Apache Tomcat 9	19
1.7.3 Step 3: Start Tomcat Server	19
1.7.4 Step 4: Verify Installation	20
1.8 Task 5: NetBeans IDE Installation	21
1.8.1 Step 1: Download Apache NetBeans 29	21
1.8.2 Step 2: Extract and Install NetBeans 29	21
1.8.3 Step 3: Configure NetBeans 29	23
1.8.4 Step 5: Configure Tomcat in NetBeans	23
1.9 Task 6: Writing a Simple Java Servlet	26
1.9.1 Step 1: Create New Project	26
1.9.2 Step 2: Create New Servlet	29
1.9.3 Step 3: Run Servlet	32
1.10 Task 7: Writing a Simple JSP Program	34
1.10.1 Step 1: Create a New Project	34
1.10.2 Step 2: Create a New JSP File	36
1.10.3 Step 3: Run JSP File	39
1.11 Task 8: Use Java Reference Datatype/Class Wrapper in JSP	41
1.11.1 Step 1: Create a New JSP File	41

1.12	Exercise	43
2	SERVLET: DATA SHARING AND DATABASE MANAGEMENT	45
2.1	Objectives	45
2.2	Introduction	46
2.3	Software Requirements	46
2.4	Task 1: Data Sharing in Servlet	46
2.4.1	Step 1: Project Setup	46
2.4.2	Step 2: Create HTML Login Page	48
2.4.3	Step 3: Create a Login Servlet	48
2.4.4	Step 4: Create an Account Servlet	51
2.4.5	Step 5: Run the Project	52
2.5	Task 2: Creating A Table in MySQL Database	53
2.5.1	Step 1: Start Apache and MySQL	53
2.5.2	Step 2: Create a New Database	53
2.5.3	Step 3: Create a New MySQL Table	53
2.6	Task 3: Setting the Environment of Web Application for Database Connection	54
2.6.1	Step 1: Create a New Web Application Project	54
2.6.2	Step 2: Prepare the MySQL Connector	55
2.7	Task 4: Using Servlets for Database CRUD Operations	57
2.7.1	Step 1: Create HTML Form for Data Entry	57
2.7.2	Step 2: Create the Insert Servlet (Create)	58
2.7.3	Step 3: Create the View Servlet (Read)	59
2.7.4	Step 4: Create the Delete Servlet (Delete)	62
2.7.5	Step 5: Create the Update Servlet (Update)	63
2.8	Task 5: Using Servlets, DAO and Java Beans for Database CRUD Operations	65
2.8.1	Step 1: Copy and Rename the Project	66
2.8.2	Step 2: Create the User JavaBean (Model)	66
2.8.3	Step 3: Create the Data Access Object (UserDAO)	68
2.8.4	Step 4: Modify the Existing Servlets (Controller)	72
2.9	Exercise	77
3	JSP: SCRIPTLET, PAGE DIRECTIVE & INCLUDE DIRECTIVE	79
3.1	Objectives	79
3.2	Introduction	79
3.3	Software Requirements	80
3.4	Task 1: Passing Data from Main JSP's Page to Other JSP's Page	80
3.4.1	Step 1: Create A New JSP Page	80
3.4.2	Step 2: Write an HTML Markup	81
3.5	Task 2: Using Mathematics Operations in JSP	84
3.5.1	Step 1: Create A New JSP Page	85
3.5.2	Step 2: Write an HTML Markup	85
3.5.3	Step 3: Run the <code>Calculator.jsp</code>	86
3.6	Task 3: Populate an Array Values into HTML's Table	89
3.6.1	Step 1: Create A New JSP Page	89
3.6.2	Step 2: Write An HTML Markup	89
3.6.3	Step 3: Run the <code>populateArray.jsp</code> File	90

3.7	Task 4: Perform Calculation of Car Loan	90
3.7.1	Step 1: Create A New HTML File	90
3.7.2	Step 2: Create An HTML Markup	91
3.7.3	Step 3: Create <code>processCalculateCarLoan.jsp</code>	92
3.8	Task 5: Using JSP Page Directive to Call Java API	93
3.9	Task 6: Use JSP Include directive for JSP Page	98
4	JSP: SCRIPTLET, EXPRESSION & STANDARD ACTIONS	103
4.1	Objectives	103
4.2	Introduction	103
4.3	Software Requirements	104
4.4	Task 1: Using JSP Scripting	110
4.4.1	Step 1: Create <code>customer.html</code>	110
4.4.2	Step 2: Create <code>processCustomer.jsp</code>	111
4.4.3	Step 3: Compile and Run the Application	113
4.5	Task 2: Using JSP (Scripting, Declaration and Expression)	115
4.5.1	Step 1: Create <code>currencyConversion.html</code>	115
4.5.2	Step 2: Create <code>processCurrency.jsp</code>	116
4.5.3	Step 3: Compile and Run the Application	118
4.6	Task 3: Using JSP Standard Action (Include and Param)	120
4.6.1	Step 1: Create <code>jspParameter.jsp</code>	120
4.6.2	Step 2: Create <code>subjectInfo.jsp</code>	122
4.6.3	Step 3: Compile and Run the Application	123
4.7	Task 4: Using JSP Standard Action (Forward)	124
4.7.1	Step 1: Create <code>forward.jsp</code>	124
4.7.2	Step 2: Create <code>forwardInfo.jsp</code>	125
4.7.3	Step 3: Compile and Run the Application	126
4.8	Task 5: Using JSP Scriptlet to Construct Business Logic	127
4.8.1	Step 1: Create Project and JSP File <code>insuranceQuotation.jsp</code>	127
4.8.2	Step 2: Design Insurance Quotation Interface	127
4.8.3	Step 3: Create <code>processInsuranceQuo.jsp</code>	128
4.8.4	Step 4: Implement Business Logic Using JSP Scriptlet	129
4.8.5	Step 5: Compile and Run the Application	130
5	JSP: JAVABEANS AND STANDARD TAG LIBRARY (JSTL)	135
5.1	Objectives	135
5.2	Introduction	136
5.3	Software Requirements	136
5.4	Task 1: Using Scriptlet to Access a Simple JavaBeans	136
5.4.1	Step 1: Create the JavaBean	136
5.4.2	Step 2: Access Bean via Scriptlet	137
5.5	Task 2: Problem Solving using JavaBeans	138
5.5.1	Step 1: Create a Calculator Bean	138
5.5.2	Step 2: Create Form and Process Page	139
5.6	Task 3: Installing JSTL Taglibs	140
5.6.1	Step 1: Download and Import JSTL Libraries	141
5.6.2	Step 2: Test the Installation and Formatting Tags	142
5.7	Task 4: Using Java Standard Tag Library (JSTL) Core Tags	144

5.7.1	Step 1: Variables and Conditionals	144
5.8	Task 5: Using JSP Standard Tag Library (JSTL) for Collections	145
5.8.1	Step 1: Create the Servlet (Controller)	145
5.8.2	Step 2: Create the View (JSTL forEach)	146
5.9	Task 6: Database Access using JSTL SQL Tags	147
5.9.1	Step 1: Create Database and Table	148
5.9.2	Step 2: Prepare the Database Driver	148
5.9.3	Step 3: Create the JSTL SQL Page	149
5.10	Exercise: Employee Payroll Display System	151

List of Figures

1.1	XAMPP Download Platform	3
1.2	XAMPP Setup Wizard Startup Screen	5
1.3	XAMPP Select Components	5
1.4	XAMPP Installation Directory	6
1.5	XAMPP Language	7
1.6	XAMPP Installation Ready	7
1.7	XAMPP Installation Progress	8
1.8	XAMPP Installation Finish	8
1.9	XAMPP Control Panel	9
1.10	XAMPP Control Panel Running	10
1.11	XAMPP Dashboard	10
1.12	phpMyAdmin Interface	11
1.13	XAMPP Control Panel-Click Shell	11
1.14	Verify Password	12
1.15	Error Page	13
1.16	Apache Tomcat9 Download Page	18
1.17	Start Tomcat Server	19
1.18	Apache Tomcat Homepage	20
1.19	Apache NetBeans 29 Download Page	21
1.20	Extract netbeans-29-bin.zip	22
1.21	Netbean Executable File	22
1.22	Netbean IDE GUI	23
1.23	Configure Server	24
1.24	Add Server	24
1.25	Add Server	25
1.26	Add Server	25
1.27	Add Server	26
1.28	Create New Project	27
1.29	Create New Web Application	27
1.30	Enter Project Details	28
1.31	Create New Servlet	29
1.32	Name the Servlet	30
1.33	Configure Servlet Deployment	30
1.34	HelloServlet.java File	31
1.35	<code>processRequest()</code> Method	31
1.36	Modified <code>processRequest()</code> Method	31
1.37	Run the Servlet	32

1.38	Dialog Box Call to /HelloServlet	32
1.39	Output	33
1.40	Modified <code>doGet()</code> Method	33
1.41	Dialog Box with Passing Parameter	34
1.42	Output	34
1.43	Create New Web Application	35
1.44	Enter Project Details	36
1.45	Configure the Server	36
1.46	Create a New JSP File	37
1.47	Enter Name and Location	37
1.48	Modified JSP File	38
1.49	Compile JSP File	38
1.50	Compile JSP File	38
1.51	Run JSP File	39
1.52	Output	40
1.53	Start/Stop Apache Tomcat	40
1.54	Create <code>useJavaObject.jsp</code>	41
1.55	<code>useJavaObject.jsp</code>	42
1.56	Output	43
2.1	Create New Servlet	49
2.2	Create Login Servlet	49
2.3	Configure servlet deployment	50
2.4	Creating the “CRUDusingServlet” Project	55
2.5	Downloading MySQL Connector/J	55
2.6	Extracting the MySQL Connector/J Archive	56
2.7	Adding External Library to Project	56
2.8	Importing MySQL Connector/J Library	57
3.1	Create new JSP	81
3.2	New JSP	81
3.3	<code>memberRegister.jsp</code> Interface	82
3.4	Filling input <code>memberRegister.jsp</code>	84
3.5	Output in <code>memberProcessing.jsp</code>	84
3.6	Create new JSP	85
3.7	New JSP	85
3.8	<code>calculator.jsp</code> Interface	86
3.9	Key-in Filename	89
3.10	Output from Array	90
3.11	Creating New HTML File	91
3.12	Interface Output for Car Loan Calculation HTML	91
3.13	Input for Car Loan Calculation	92
3.14	Output for Car Loan Calculation	93
3.15	New JSP	93
3.16	Key-in Filename	94
3.17	Save All	96
3.18	Run	96
3.19	Success Message	96

3.20	ArrayList Output	97
3.21	Tomcat Before Run	97
3.22	Tomcat After Run	97
3.23	Tomcat Success Message	98
3.24	mainPage.jsp	98
3.25	headerPage.jsp Output	98
3.26	footerPage.jsp Output	99
3.27	mainPage.jsp Output	99
4.1	Creating customer.html	111
4.2	Interface of customer.html	111
4.3	Creating processCustomer.jsp	112
4.4	Example input in customer.html	114
4.5	Output after processing customer input	114
4.6	Creating currencyConversion.html	115
4.7	Interface of currencyConversion.html	116
4.8	Creating processCurrency.jsp	116
4.9	Interface of currencyConversion.html	119
4.10	Input example in currencyConversion.html	119
4.11	Processing result of currency conversion	120
4.12	Creating jspParameter.jsp	121
4.13	Creating subjectInfo.jsp	122
4.14	Output of jspParameter.jsp	123
4.15	Creating forward.jsp	124
4.16	Creating forwardInfo.jsp	125
4.17	Output of forwarded information	126
4.18	Creating insuranceQuotation.jsp	127
4.19	Insurance Quotation Interface	128
4.20	Creating processInsuranceQuo.jsp	128
4.21	Insurance Quotation Output	131
5.1	Download jstl-1.2.jar	141

Lab Module 5

JSP: JAVABEANS AND STANDARD TAG LIBRARY (JSTL)

5.1 Objectives

By the end of this lab, students should be able to:

1. Instantiate and access simple JavaBeans within JSP pages using standard scriptlets and expressions.
2. Utilize JSP Action Tags such as `<jsp:useBean>`, `<jsp:setProperty>`, and `<jsp:getProperty>` to process HTML form data automatically.
3. Download, install, and configure the JavaServer Pages Standard Tag Library (JSTL) in a web application project.
4. Implement JSTL Core Tags (e.g., `<c:set>`, `<c:out>`, `<c:if>`) to perform variable declarations, value output, and conditional checks without writing Java code inside HTML.
5. Iterate through collections of objects and display dynamic data using the JSTL `<c:forEach>` loop combined with JavaBeans.
6. Design and develop structured web applications by enforcing a clean separation between business logic (Model) and presentation (View).

5.2 Introduction

This lab introduces the concepts of **JavaBeans** and the **JavaServer Pages Standard Tag Library (JSTL)** to promote a clean separation of business logic from presentation in dynamic web applications. Students will explore how JavaBeans encapsulate data and methods, and how they can be seamlessly accessed within JSP pages using both standard scriptlets and JSP action tags (`<jsp:useBean>`, `<jsp:setProperty>`, `<jsp:getProperty>`).

In addition, the lab emphasizes replacing cumbersome Java scriptlets in HTML with clean, concise **JSTL Core Tags** (`<c:set>`, `<c:if>`, `<c:forEach>`). By removing direct Java code from JSP pages, the application becomes easier to read, maintain, and debug. Through a series of structured tasks, students will progressively build their skills from instantiating simple beans to iterating over collections of objects, ultimately developing structured and reusable JSP applications that adhere closely to modern web development practices.

5.3 Software Requirements

Software / Library	Website
Java JDK	https://www.oracle.com/java/technologies/downloads/
XAMPP	https://www.apachefriends.org
Apache Tomcat	https://tomcat.apache.org
JSTL 1.2 / Jakarta Taglibs	https://mvnrepository.com/artifact/jstl/jstl/1.2

5.4 Task 1: Using Scriptlet to Access a Simple JavaBeans

Objective: To understand how to create a basic JavaBean and instantiate it inside a JSP page using standard Java scriptlets.

Estimated time: 20 minutes

5.4.1 Step 1: Create the JavaBean

1. In your NetBeans project, create a new Java Class named `StudentBean` inside the `com.lab.bean` package.

2. Write the following code to declare private variables and generate the standard Getter and Setter methods.

```
package com.lab.bean;

public class StudentBean {
    private String name;
    private String matricNo;

    // Empty Default Constructor (Compulsory for JavaBeans)
    public StudentBean() {
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getMatricNo() {
        return matricNo;
    }

    public void setMatricNo(String matricNo) {
        this.matricNo = matricNo;
    }
}
```

5.4.2 Step 2: Access Bean via Scriptlet

1. Create a new JSP file named `task1.jsp` inside your Web Pages folder.
2. Write the following code to import the bean, set values using a scriptlet, and print them using an expression tag.

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="com.lab.bean.StudentBean"%>
<!DOCTYPE html>
```

```

<html>
<head>
    <title>Task 1: Scriptlet & Bean</title>
</head>
<body>
    <h2>Student Information (Using Scriptlet)</h2>

    <%
        // Instantiating the bean
        StudentBean student = new StudentBean();
        student.setName("Ahmad Ali");
        student.setMatricNo("S123456");
    %>

    <p><strong>Name:</strong> <%= student.getName() %></p>
    <p><strong>Matric No:</strong> <%= student.getMatricNo() %></p>
</body>
</html>

```

Reflections

1. What is the main disadvantage of mixing Java scriptlets directly inside HTML codes?
2. Why is it compulsory for a JavaBean class to have an empty default constructor?

5.5 Task 2: Problem Solving using JavaBeans

Objective: To utilize JSP standard action tags (<jsp:useBean>, <jsp:setProperty>) to process HTML form data automatically without using scriptlets.

Estimated time: 30 minutes

5.5.1 Step 1: Create a Calculator Bean

1. Create a new Java Class named CalculatorBean in the com.lab.bean package.
2. Declare two double variables and add a custom getSum() method.

```

package com.lab.bean;

public class CalculatorBean {
    private double number1;
    private double number2;

    public CalculatorBean() {
    }

    public double getNumber1() {
        return number1;
    }

    public void setNumber1(double number1) {
        this.number1 = number1;
    }

    public double getNumber2() {
        return number2;
    }

    public void setNumber2(double number2) {
        this.number2 = number2;
    }

    // Custom method for logic
    public double getSum() {
        return number1 + number2;
    }
}

```

5.5.2 Step 2: Create Form and Process Page

1. Create `calculator.html` with a form that asks for two numbers. Notice that the `name` attributes match the variables in our Bean exactly.

```

<!DOCTYPE html>
<html>
<body>

```

```

<h2>Simple Calculator</h2>
<form action="process.jsp" method="POST">
    Number 1: <input type="text" name="number1"><br><br>
    Number 2: <input type="text" name="number2"><br><br>
    <input type="submit" value="Calculate">
</form>
</body>
</html>

```

2. Create `process.jsp` to capture the data using JSP Action Tags automatically.

```

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<body>
    <h2>Calculation Result</h2>

    <jsp:useBean id="calc" class="com.lab.bean.CalculatorBean" />

    <jsp:setProperty name="calc" property="*" />

    <p>The sum of
        <jsp:getProperty name="calc" property="number1" /> and
        <jsp:getProperty name="calc" property="number2" /> is:
        <strong><jsp:getProperty name="calc" property="sum" /></strong>
    </p>

    <br><a href="calculator.html">Calculate Again</a>
</body>
</html>

```

Reflections

1. Explain how the `property="*" attribute works in the <jsp:setProperty> tag.`
2. How do JSP action tags improve code readability compared to the scriptlets used in Task 1?

5.6 Task 3: Installing JSTL Taglibs

Objective: To download, install, configure, and test the JavaServer Pages Standard Tag Library (JSTL) in a web project.

Estimated time: 15 minutes

5.6.1 Step 1: Download and Import JSTL Libraries

1. Download the JSTL 1.2 JAR file from the official Maven repository at <https://mvnrepository.com/artifact/javax.servlet/jstl/1.2>.

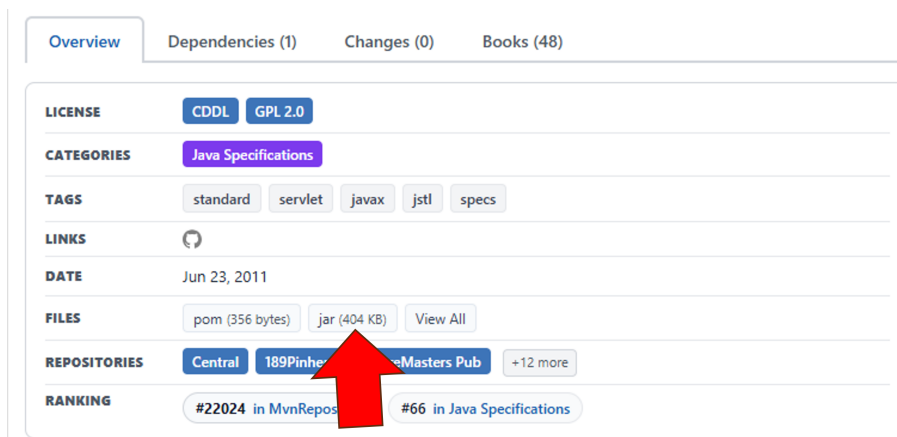


Figure 5.1: Download jstl-1.2.jar

ATTENTION (For Tomcat 10+ Users):

If you are using **Apache Tomcat 10 or newer**, the older javax JSTL library will not work. You must download the **Jakarta Standard Tag Library (JSTL 3.0)** instead.

URL: <https://mvnrepository.com/artifact/jakarta.servlet.jsp.jstl/jakarta.servlet.jsp.jstl-api>

2. In the NetBeans **Projects** window, right-click on the **Libraries** folder of your current project.
3. Select **Add JAR/Folder...** and locate the downloaded JSTL JAR file. Click **Open**.
4. Create a new JSP file named `jstl_test.jsp`.
5. Add the JSTL Core and Format taglib directives at the very top of your file (before the HTML tag):

Note on URI:

If using **Tomcat 9 or older**, use:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

If using **Tomcat 10 or newer (Jakarta)**, use:

```
<%@ taglib uri="jakarta.tags.core" prefix="c" %>
```

6. In the NetBeans **Projects** window, right-click on the **Libraries** folder of your current project.
7. Select **Add JAR/Folder...** and locate the downloaded JSTL JAR file. Click **Open**.
8. Create a new JSP file named `jstl_test.jsp`.
9. Add the JSTL Core and Format taglib directives at the very top of your file (before the HTML tag):

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/fmt" prefix="fmt" %>
```

5.6.2 Step 2: Test the Installation and Formatting Tags

1. Inside the `<body>` of your `jstl_test.jsp`, write the following code. We will declare a raw number using the Core tag (`<c:set>`) and format it as a currency using the Format tag (`<fmt:formatNumber>`). We will also retrieve the current date using `<jsp:useBean>` and format it using `<fmt:formatDate>`.

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/fmt" prefix="fmt" %>
<!DOCTYPE html>
<html>
<head>
  <title>JSTL Date & Time Formatting</title>
</head>
<body>
  <h2>Testing JSTL Date & Time Formatting</h2>

  <jsp:useBean id="now" class="java.util.Date" />
```

```

<p>1. Date Only (Default):
    <strong>
        <fmt:formatDate value="\${now}" type="date" />
    </strong>
</p>

<p>2. Time Only (Default):
    <strong>
        <fmt:formatDate value="\${now}" type="time" />
    </strong>
</p>

<p>3. Both Date & Time:
    <strong>
        <fmt:formatDate value="\${now}" type="both" />
    </strong>
</p>

<p>4. Long Date & Short Time:
    <strong>
        <fmt:formatDate value="\${now}" type="both" dateStyle="long"
            timeStyle="short" />
    </strong>
</p>

<p>5. Custom Pattern (dd MMM yyyy, hh:mm a):
    <strong>
        <fmt:formatDate value="\${now}" pattern="dd MMMM yyyy, hh:mm:ss a" />
    </strong>
</p>

</body>
</html>

```

2. Right-click `jstl_test.jsp` in the Projects window and select **Run File**.

3. If the browser successfully displays the lists of beautifully formatted numbers (such as **RM 15,230.75** and **15%**) along with the dynamically generated current date and time, your JSTL installation is correct and fully functioning!

Reflections

1. What is the purpose of the `uri` and `prefix` attributes in the `taglib` directive?
2. Why do we need separate `taglib` directives for `core` and `fmt`?

5.7 Task 4: Using Java Standard Tag Library (JSTL) Core Tags

Objective: To perform variable declarations, value output, and conditional checks using JSTL Core Tags.

Estimated time: 20 minutes

5.7.1 Step 1: Variables and Conditionals

1. Open the `jstl_test.jsp` file created in Task 3.
2. Write the following code to implement variable creation (`c:set`), output display (`c:out`), and basic conditional logic (`c:if`).

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE html>
<html>
<head>
  <title>JSTL Core Tags</title>
</head>
<body>
  <h2>JSTL Conditional Test</h2>

  <c:set var="score" value="85" />

  <p>Student Score: <c:out value="${score}" /> marks.</p>

  <c:if test="${score >= 50}">
    <p style="color: green;"><strong>Status: Pass! Congratulations.</strong>
  </p>
</c:if>

  <c:if test="${score < 50}">
```

```
<p style="color: red;"><strong>Status: Fail. Please try again.</strong>
</p>
</c:if>

</body>
</html>
```

Reflections

1. How does the `<c:out>` tag help in preventing Cross-Site Scripting (XSS) attacks?
2. The `<c:if>` tag does not have an "else" statement. Which JSTL tags should be used if you need an if-else logic structure?

5.8 Task 5: Using JSP Standard Tag Library (JSTL) for Collections

Objective: To combine JavaBeans and JSTL by iterating through a collection of objects using the `<c:forEach>` loop.

Estimated time: 30 minutes

5.8.1 Step 1: Create the Servlet (Controller)

1. Create a Servlet named `StudentListServlet` inside the `com.lab.controller` package.
2. Write the code below to populate a List of `StudentBean` objects and forward it to a JSP file.

```
package com.lab.controller;

import com.lab.bean.StudentBean;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
```

```

import javax.servlet.http.HttpServletResponse;

public class StudentListServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse
        response) throws ServletException, IOException {

        // Create a list to hold student objects
        List<StudentBean> stdList = new ArrayList<>();

        // Adding sample data
        StudentBean s1 = new StudentBean();
        s1.setName("Ali bin Abu"); s1.setMatricNo("S111");

        StudentBean s2 = new StudentBean();
        s2.setName("Siti Aminah"); s2.setMatricNo("S222");

        StudentBean s3 = new StudentBean();
        s3.setName("Muthusamy"); s3.setMatricNo("S333");

        stdList.add(s1);
        stdList.add(s2);
        stdList.add(s3);

        // Share data with JSP
        request.setAttribute("listData", stdList);

        // Forward to View
        RequestDispatcher rd = request.getRequestDispatcher("displayList.jsp");
        rd.forward(request, response);
    }
}

```

5.8.2 Step 2: Create the View (JSTL forEach)

1. Create `displayList.jsp` and write the code below to loop through the `listData` dynamically using `<c:forEach>`.
2. **Important:** Run the `StudentListServlet.java` file to test this application, **not** the JSP file!

```

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE html>
<html>
<head>
    <title>JSTL Loop Collection</title>
</head>
<body>
    <h2>Registered Students List</h2>

    <table border="1" cellpadding="8">
        <thead>
            <tr style="background-color: lightgray;">
                <th>No.</th>
                <th>Name</th>
                <th>Matric Number</th>
            </tr>
        </thead>
        <tbody>
            <c:forEach items="${listData}" var="student" varStatus="status">
                <tr>
                    <td>${status.count}</td>
                    <td>${student.name}</td>
                    <td>${student.matricNo}</td>
                </tr>
            </c:forEach>
        </tbody>
    </table>
</body>
</html>

```

Reflections

1. Describe how the `items` and `var` attributes work together in the `<c:forEach>` tag.
2. Why is it important to run the Servlet instead of the JSP file directly in this task?

5.9 Task 6: Database Access using JSTL SQL Tags

Objective: To connect to a MySQL database and execute SQL queries directly from a JSP page using the JSTL SQL Tag Library.

Estimated time: 25 minutes

Important Note: While JSTL SQL tags are incredibly powerful for rapid prototyping, using them in large applications is generally discouraged as it breaks the MVC architecture by putting database logic (Model) directly into the presentation layer (View).

5.9.1 Step 1: Create Database and Table

1. Open your database management tool (e.g., phpMyAdmin from your XAMPP control panel).
2. Open the **SQL** tab and execute the following commands to create a new database schema named **webappdevlab5**, select it, create a **users** table, and insert some sample data.

```
-- 1. Create the database schema
CREATE DATABASE webappdevlab5;

-- 2. Select the database
USE webappdevlab5;

-- 3. Create the users table
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL,
    roles VARCHAR(50) NOT NULL
);

-- 4. Insert sample data into the table
INSERT INTO users (username, roles) VALUES ('admin', 'Administrator');
INSERT INTO users (username, roles) VALUES ('johndoe', 'Staff');
INSERT INTO users (username, roles) VALUES ('janedoe', 'Manager');
```

5.9.2 Step 2: Prepare the Database Driver

1. Ensure that the **MySQL JDBC Driver** (`mysql-connector-java.jar`) is added to your NetBeans project's **Libraries** folder. JSTL needs this driver to communicate with the MySQL database.

5.9.3 Step 3: Create the JSTL SQL Page

1. Create a new JSP file named `jstl_database.jsp`.
2. Add both the JSTL Core and SQL taglib directives at the top of the file.
3. Use `<sql:setDataSource>` to configure the database connection (make sure the URL points to `webappdevlab5`).
4. Use `<sql:query>` to retrieve data from the `users` table.
5. Use `<c:forEach>` to loop through the `result.rows` and display the data.

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/sql" prefix="sql" %>
<!DOCTYPE html>
<html>
<head>
  <title>JSTL SQL Tags</title>
</head>
<body>
  <h2>User List (Fetched directly using JSTL SQL)</h2>

  <sql:setDataSource var="dbConnection" driver="com.mysql.cj.jdbc.Driver"
    url="jdbc:mysql://localhost:3306/webappdevlab5"
    user="root" password="" />

  <sql:query dataSource="${dbConnection}" var="result">
    SELECT * FROM users;
  </sql:query>

  <table border="1" cellpadding="8">
    <thead>
      <tr style="background-color: lightblue;">
        <th>ID</th>
        <th>Username</th>
        <th>Roles</th>
      </tr>
    </thead>
    <tbody>
      <c:forEach var="row" items="${result.rows}">
```

```
        <tr>
            <td><c:out value="${row.id}" /></td>
            <td><c:out value="${row.username}" /></td>
            <td><c:out value="${row.roles}" /></td>
        </tr>
    </c:forEach>
</tbody>
</table>

</body>
</html>
```

6. Run `jstl_database.jsp`. You should see the records from your database printed gracefully in the HTML table without writing a single line of Java code!

Reflections

1. The JSTL `<sql:setDataSource>` tag allows you to retrieve data without writing Servlets or DAO classes. Why is this considered a bad practice (violating the MVC pattern) for large-scale enterprise applications?

5.10 Exercise: Employee Payroll Display System

Based on the concepts you have learned in this lab (JavaBeans, Servlets as Controllers, and JSTL for the View), you are required to independently develop a simple **Employee Payroll Display System**.

Please complete this exercise by fulfilling the following requirements:

1. **JavaBean (Model):** Create a class named `Employee` in the `com.lab.bean` package with the following private attributes: `empId` (String), `name` (String), `department` (String), and `basicSalary` (double). Include the standard empty constructor and the getter/setter methods for all attributes.
2. **Servlet (Controller):** Create a `PayrollServlet` in the `com.lab.controller` package. Inside this Servlet's `doGet()` method:
 - Create a list (`ArrayList`) and populate it with at least 5 different `Employee` objects.
 - Share this list to the JSP using `request.setAttribute("employeeList", list);`.
 - Forward the request to a view file named `payroll_view.jsp`.
3. **JSP & JSTL (View):** Create `payroll_view.jsp` to display the data:
 - Import the JSTL core taglib directive at the very top of the file.
 - Use the `<c:forEach>` tag to iterate through the `employeeList` and display their details in a well-formatted HTML table.
 - **Logic Challenge:** Use the `<c:choose>`, `<c:when>`, and `<c:otherwise>` tags inside the table to add a new "Status" column. If the employee's basic salary is RM3000 or above, display the text "Senior"; otherwise, display "Junior".

Assessment Rubric (10 Marks)

Criteria	Marks
JavaBean Creation (2 Marks): Correct implementation of the <code>Employee</code> class with private attributes, empty constructor, and standard getters/setters.	2
Controller Implementation (3 Marks): Proper instantiation of multiple <code>Employee</code> objects, adding them to a List, and correctly forwarding the request attributes.	3
JSTL Iteration (3 Marks): Correct use of the JSTL directive and the <code><c:forEach></code> tag to populate the HTML table dynamically.	3
JSTL Conditional Logic (2 Marks): Successful implementation of <code><c:choose></code> tags to evaluate the salary and display the correct employment status.	2
Total	10